

EXPERIMENT – 12.2

Name: K. ANSHU
Roll No: 2520030440

Title

Deadlock Prevention Using Resource Ordering

Aim

To demonstrate deadlock prevention by acquiring shared resources in a fixed order using POSIX threads and mutex locks.

Theory

Deadlock can occur when threads wait indefinitely for resources held by one another. Resource ordering prevents this situation by requiring all threads to acquire resources in the same predefined order. In this program, both threads lock Resource 1 before Resource 2, then release the locks after use.

Algorithm

1. Declare two mutex locks representing Resource 1 and Resource 2.
2. Initialize both mutexes.
3. Create two threads.
4. In each thread, acquire Resource 1 first.
5. Acquire Resource 2 after Resource 1 and display the acquired resources.
6. Release Resource 2 and then Resource 1.
7. Join both threads and destroy the mutexes.
8. Display the completion message.

Program Code

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 1 acquired Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 1 acquired Resource 2\n");
```

```

        pthread_mutex_unlock(&resource2);
        pthread_mutex_unlock(&resource1);

        return NULL;
    }

void *thread2(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 2 acquired Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 2 acquired Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Deadlock prevention using resource ordering completed.\n");

    return 0;
}

```

Execution and Screenshots

Screenshot 1 – Thread Functions

```
admin@heeraj@DESKTOP-J39GHP7:~$ cat > exp12_deadlock.c <<'EOF'
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 1 acquired Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 1 acquired Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 2 acquired Resource 1\n");

    sleep(1);
```

Screenshot 2 – Main Function

```

    pthread_mutex_lock(&resource2);
    printf("Thread 2 acquired Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Deadlock prevention using resource ordering completed.\n");

    return 0;
}

```

Screenshot 3 – Compilation and Execution Output

```

dmindheeraj@DESKTOP-J39GHP7:~$ gcc exp12_deadlock.c -o exp12_deadlock -pthread
dmindheeraj@DESKTOP-J39GHP7:~$ ./exp12_deadlock
Thread 1 acquired Resource 1
Thread 1 acquired Resource 2
Thread 2 acquired Resource 1
Thread 2 acquired Resource 2
Deadlock prevention using resource ordering completed.
dmindheeraj@DESKTOP-J39GHP7:~$ |

```

Observed Output

```

Thread 1 acquired Resource 1
Thread 1 acquired Resource 2
Thread 2 acquired Resource 1
Thread 2 acquired Resource 2
Deadlock prevention using resource ordering completed.

```

Result

Deadlock prevention using resource ordering was demonstrated successfully. Both threads acquired the resources in the same order, completed their execution, and displayed the completion message.